



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

КУРСОВАЯ РАБОТА

по дисциплине: Шаблоны программных платформ языка Джава

по профилю: Разработка программных продуктов и проектирование информационных систем

направления профессиональной подготовки: 09.03.04 «Программная инженерия»

Тема: Приложение «Ресторан»

Студент: Никитина В.А.

Группа: ИКБО-32-21

Работа представлена к защите 08.10.2024 (дата) А.Н. /Никитина В.А./
(подпись и ф.и.о. студента)

Руководитель: старший преподаватель кафедры ИиППО Рачков Андрей Владимирович

Работа допущена к защите 08.10.2024 (дата) Рачков А.В. /Рачков А.В./
(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: удовл.

Д.А. /к.т.н., доц. Куликов В.А.А. /
08.10.24 /ст. преп. Рачков А.В. /

(подписи, дата, ф.и.о., должность, звание, уч. степень двух преподавателей, принявших защиту)

М. РТУ МИРЭА. 2023 г.



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ

на выполнение курсовой работы

по дисциплине: Шаблоны программных платформ языка Джава
по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Никитина Валерия Александровна

Группа: ИКБО-32-21

Срок представления к защите: 11.05.2023 г.

Руководитель: старший преподаватель Рачков Андрей Владимирович

Тема: Приложение «Ресторан»

Исходные данные: индивидуальное задание на разработку; документация по Spring Framework и JEE, документация по языку Java (версия не ниже 8); инструменты и технологии: JDK (не ниже 8), создание Spring MVC web-приложений, RESTful web-сервисов, Spring ORM и Spring DAO, Gradle, gitHub, IntelliJIDEA. Нормативный документ: инструкция по организации и проведению курсового проектирования СМКО МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. Провести анализ предметной области и формирование основных требований к приложению.
2. Обосновать выбор средств ведения разработки. 3. Разработать приложение с использованием фреймворка Spring, выбранной технологии и инструментария. 4. Провести тестирование приложения. 5. Оформить пояснительную записку по курсовой работе в соответствии с ГОСТ 7.32-2017. 6. Провести анализ текста на антиплагиат 7. Создать презентацию по выполненной курсовой работе.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: [подпись] /Р. Г. Болбаков/, « 20 » 02 2023 г.

Задание на КР выдал: [подпись] / А.В. Рачков/, « 20 » 02 2023 г.

Задание на КР получил: [подпись] /В.А. Никитина/, « 20 » 02 2023 г.

РЕФЕРАТ

Проект курсовой работы содержит 22 страницы, 18 рисунков, 15 информационных источников.

РАЗРАБОТКА, ВЕБ-ПРИЛОЖЕНИЕ, SPRING, РЕСТОРАН

Объект исследования: – представление работы ресторана.

В ходе проекта была разработана веб-система, основанная на архитектуре MVC, обеспечивающая взаимодействие между клиентом и серверной базой данных с использованием RESTful API.

Проект начался с анализа предметной области и исследования существующих веб-приложений, чтобы определить лучшие практики и современные решения для серверной части.

Результатом проекта стало веб-приложение, отвечающее требованиям современного сайта ресторана, включая функциональность и тематику.

The project of the course work contains 22 pages, 18 drawings, 15 information sources.

DEVELOPMENT, WEB APPLICATION, SPRING, RESTAURANT

Object of research: – representation of restaurant work.

The project involved the development of a web system based on the MVC architecture, providing interaction between the client and the server database using RESTful API.

The project began with an analysis of the subject area and a study of existing web applications to determine best practices and modern solutions for the server side.

The result of the project was a web application that meets the requirements of a modern restaurant website, including functionality and theme.

СОДЕРЖАНИЕ

РЕФЕРАТ	4
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	6
ВВЕДЕНИЕ	7
1 ОБЩИЕ СВЕДЕНИЯ	9
1.1 Анализ предметной области	9
1.2 Функциональное назначение	10
1.3 Обоснование выбора средств ведения разработки	10
2 ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ	13
2.1 Разработка клиентской части	13
2.2 Разработка моделей, репозитория, сервисов	14
2.3 Разработка контроллеров и работа с Thymeleaf	16
3 ТЕСТИРОВАНИЕ	19
ЗАКЛЮЧЕНИЕ	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете применяют следующие сокращения и обозначения.

DAO	– Data Access Object (Объект доступа к данным)
CSS	– Cascading Style Sheets (Каскадные таблицы стилей)
HTML	– HyperText Markup Language (Язык разметки гипертекста)
JSON	– JavaScript Object Notation (Нотация объектов JavaScript)
MVC	– Model-View-Controller (Модель-Представление-Контроллер)
ORM	– Object-Relational Mapping (Отображение объектно-ориентированных структур на реляционные данные)
REST	– REpresentational State Transfer (Архитектурный стиль передачи данных через HTTP)
Thymeleaf	– Java template engine (Шаблонизатор для Java)

ВВЕДЕНИЕ

Современные технологии открывают новые возможности для создания функциональных веб-приложений, которые интегрируются в различные сферы нашей жизни. Благодаря появлению новых фреймворков, мы получаем доступ к широкому спектру инструментов и решений для разработки веб-сервисов, обеспечивающих взаимодействие с пользователем, включая хранение данных, авторизацию и многое другое.

Данная курсовая работа посвящена разработке веб-приложения для ресторана, демонстрирующего преимущества использования современных технологий в сфере гостеприимства. Цель проекта - создать интуитивный и функциональный сайт, который позволит пользователям легко познакомиться с меню, сделать заказ, а также получать информацию о ресторане.

В рамках разработки приложения были выполнены следующие этапы:

- 1) анализ предметной области: изучение потребностей ресторана и определение необходимых функций веб-приложения;
- 2) выбор технологии: выбор наиболее подходящего фреймворка и инструментов для реализации проекта, в том числе для взаимодействия с базой данных, обработки запросов и обеспечения безопасности;
- 3) разработка дизайна: создание привлекательного и интуитивно понятного интерфейса с использованием HTML и CSS;
- 4) разработка серверной логики: использование Spring фреймворка для реализации функциональности веб-приложения, обработки запросов и взаимодействия с базой данных;
- 5) объединение клиентской и серверной частей: интеграция интерфейса с серверной логикой для создания целостного веб-приложения;
- 6) тестирование: проверка работоспособности и оптимизация приложения.

Итогом курсовой работы является готовое веб-приложение, которое представляет собой полноценный сайт ресторана, обладающий необходимым

функционалом для взаимодействия с пользователями и обеспечивающий удобство в получении информации о заведении.

1 ОБЩИЕ СВЕДЕНИЯ

1.1 Анализ предметной области

Для проведения анализа предметной были рассмотрены несколько аналогов, подходящих под тематику «Ресторан». Для примера были: whiterabbitmoscow.ru (Рисунок 1), bardak.cafe (Рисунок 2), zodiacmoscow.ru (Рисунок 3).

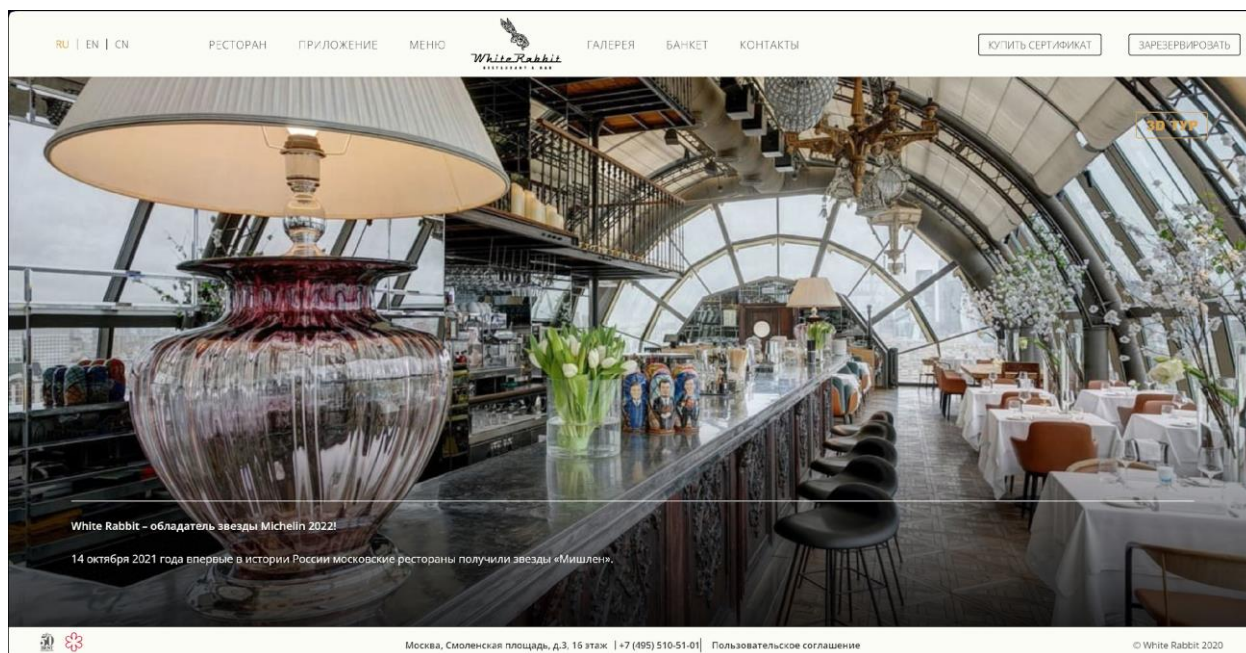


Рисунок 1 – Сайт whiterabbitmoscow.ru

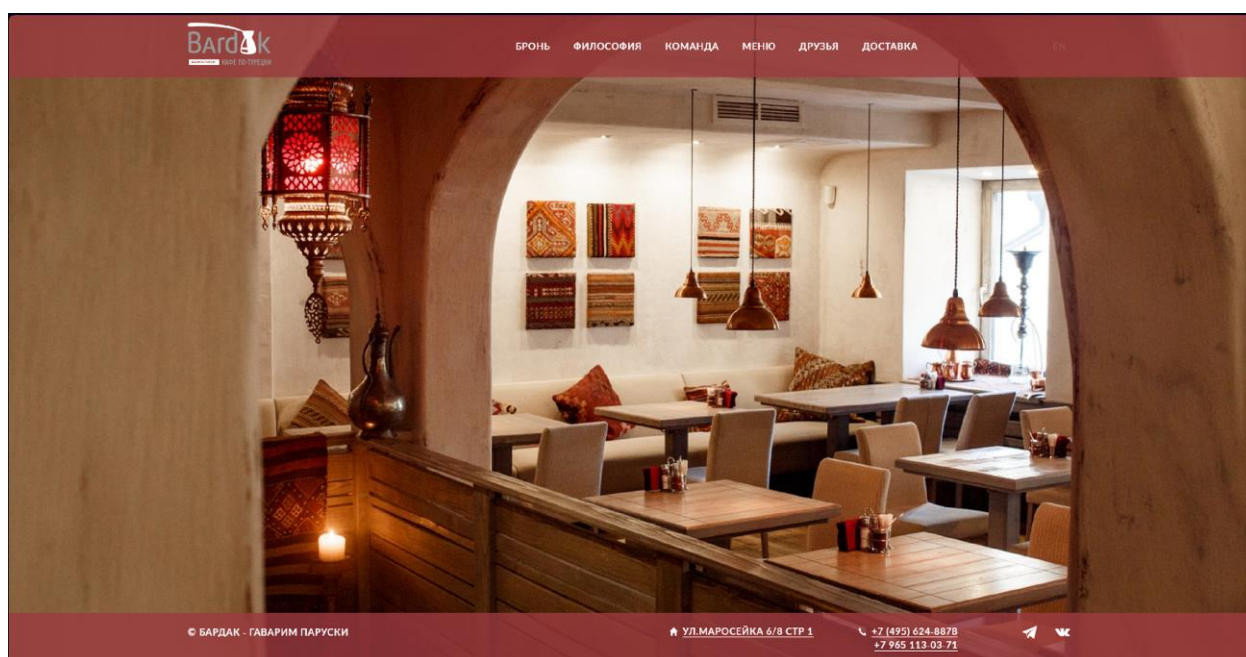


Рисунок 2 – Сайт bardak.cafe

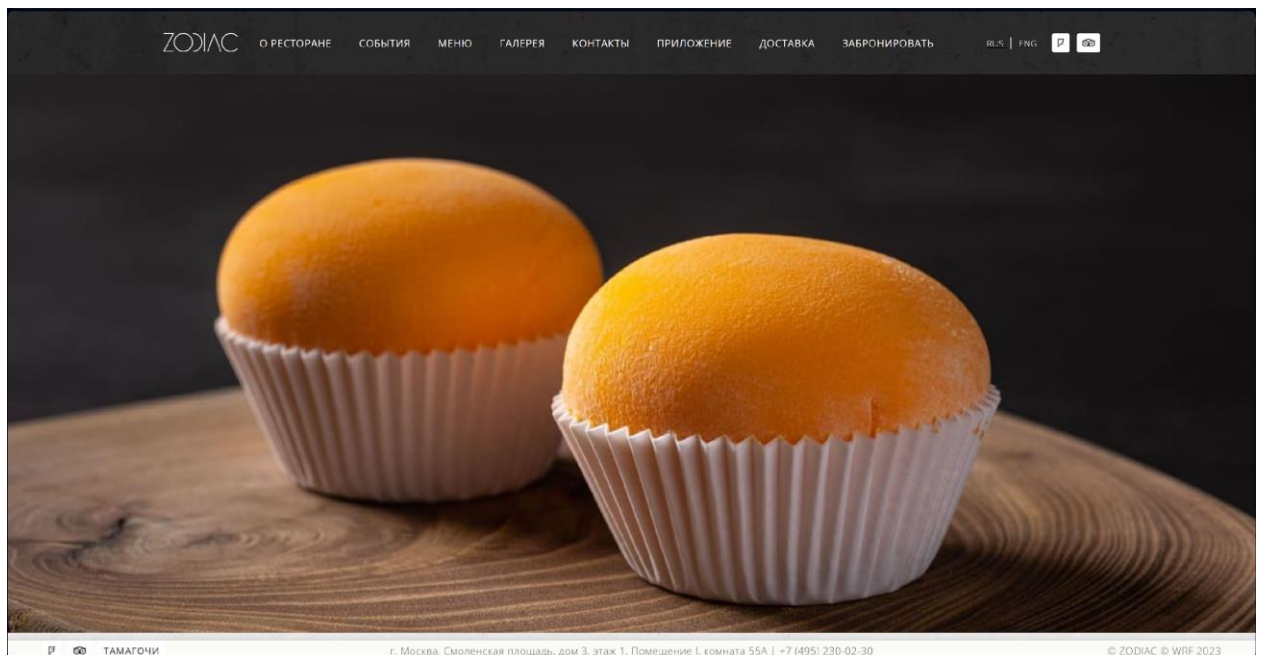


Рисунок 3 – Сайт zodiacmoscow.ru

Проанализировав представленные ресурсы, было выяснено, что на всех сайтах меню представлено в виде PDF-страницы без визуального представления. Так же на сайтах отсутствует возможность оформлять заказ онлайн, нет возможности автоматического подсчета суммы заказа.

По итогам сравнения и анализа данных интернет-ресурсов были выведены ключевые моменты, которые должно содержать веб-приложение, а именно: взаимодействие с блюдами, взаимодействие с добавлением и удалением из корзины, обновление цен на блюда, подсчет общей суммы заказа. По исследуемым данным была спланирована дальнейшая работа над серверным функционалом веб-приложения.

1.2 Функциональное назначение

Назначением является веб-приложение на тематику «Ресторан». Основной функционал позволяет клиенту получать актуальную информацию о блюдах и общей стоимости корзине благодаря серверной части. Администратор может также добавлять, обновлять, удалять записи о блюдах, что сразу будет отображаться на главной странице.

1.3 Обоснование выбора средств ведения разработки

Для разработки приложения была выбрана интегрированная среда разработки IntelliJ IDEA, а также применены технологии ORM, DAO,

Thymeleaf и MVC. Каждый из этих компонентов был тщательно выбран для оптимизации процесса разработки, повышения качества кода и упрощения дальнейшего обслуживания приложения.

IntelliJ IDEA — это одна из самых популярных и мощных IDE для разработки на Java и других языках. Она предоставляет обширный набор инструментов и функций, которые значительно упрощают процесс разработки:

- интуитивно понятный интерфейс: удобный пользовательский интерфейс и хорошо организованные меню делают разработку более комфортной;

- подсветка синтаксиса: умная подсветка и автоматическое завершение кода помогают избежать синтаксических ошибок и ускоряют написание кода;

- поддержка фреймворков: IntelliJ имеет отличную поддержку популярных фреймворков, таких как Spring и Hibernate, что облегчает разработку веб-приложений;

- инструменты для отладки: встроенные инструменты отладки позволяют быстро находить и исправлять ошибки, что экономит время разработки.

ORM позволяет разработчикам работать с базами данных в терминах объектно-ориентированного программирования, упрощая взаимодействие с данными:

- отказ от написания SQL: ORM автоматически генерирует SQL-запросы на основе операций с объектами, что устраняет необходимость вручную писать запросы. Это снижает риск ошибок и увеличивает продуктивность;

- упрощенный доступ к данным: разработчики могут использовать привычные концепции, такие как объекты и классы, для доступа к данным, что делает код более читаемым и удобным для понимания.

Шаблон проектирования DAO отделяет логику приложения от логики доступа к данным, что делает код более гибким и модульным:

- разделение логики: используя DAO, можно четко отделить бизнес-логику от логики работы с базой данных, что делает приложение более гибким и легким для изменений;

- легкость в поддержке: если нужно изменить способ доступа к данным (например, перейти на другую базу данных), достаточно обновить только DAO, не затрагивая остальной код приложения.

Thymeleaf является мощным шаблонизатором, который отлично подходит для создания динамических веб-страниц. Он предоставляет гибкие возможности для вставки и обработки данных на веб-страницах, что упрощает разработку приложений.

Pattern MVC разделяет приложение на три компонента, что делает его более структурированным:

- упрощение управления кодом: поскольку каждый компонент управляет отдельной частью приложения, это значительно облегчает поддержку и расширение функциональности;

- четкое разделение обязанностей: MVC помогает отделить модели (данные) от представлений (интерфейс) и контроллеров (логика управления), что упрощает работу команды разработчиков;

- поддерживаемая архитектура: этот шаблон интуитивно понятен и широко используется в разработке веб-приложений, что делает его полезным для обучения и масштабирования.

2 ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

2.1 Разработка клиентской части

Разработка клиентской части началась со сбора информации о других ресторанах, была выведена общая структура, которая объединяет все сайты, а именно реализации меню для клиента на Рисунке 4 и корзина с их общей стоимостью на Рисунке 5. Страница была сделана с помощью HTML [1] и CSS [2].

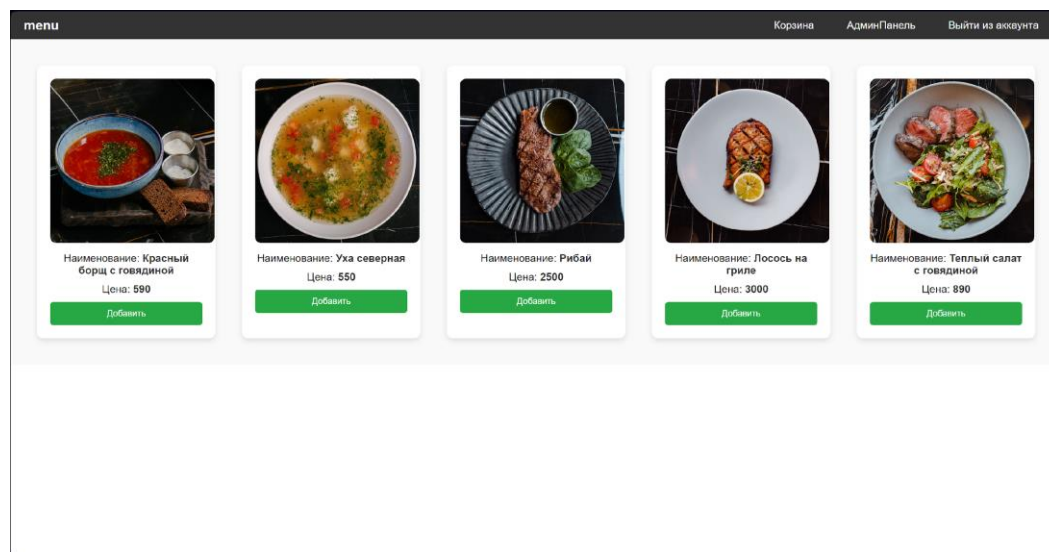


Рисунок 4 – Главная страница

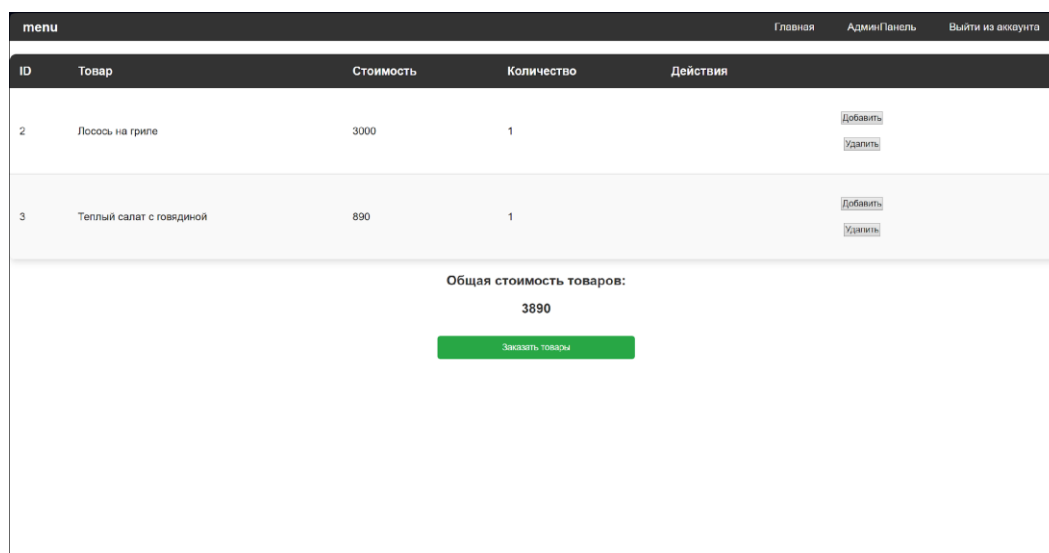


Рисунок 5 – Страница с корзиной

Также была реализована возможность для управления блюдами, что больше подходит для администрации сайта, результаты можно увидеть на Рисунке 6.

Рисунок 6 – Функционал для администрации

2.2 Разработка моделей, репозиториев, сервисов

Для реализации работы клиентской части использовался архитектурный паттерн проектирования MVC [3]. Были реализованы модели Basket и Product, которые отвечают за две БД с данными о корзине с блюдами, представленными для заказа. Чтобы получать все get и set методы для всех полей класса был использован Lombok [4], который позволяет сократить время на создание данных методов. Для реализации БД было выбрано PostgreSQL [5], Рисунок 7.

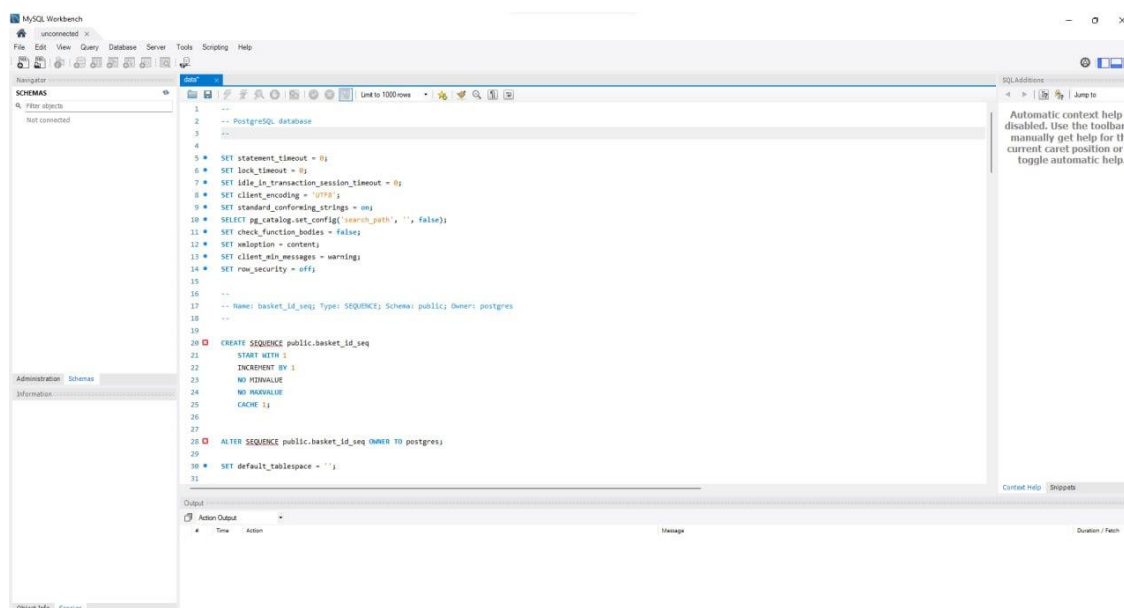


Рисунок 7 – База данных PostgreSQL

Создание моделей Basket (Рисунок 8) и Product происходило с использованием технологии ORM [6], для автоматизированной работы с БД.

```
1 package com.example.restore.Model;
2
3 > import ...
4
5
6
7
8
9 @Data 17 usages
10 @AllArgsConstructor
11 @NoArgsConstructor
12 @Entity
13 @Table(name = "basket")
14 public class Basket
15 {
16     @Id no usages
17     @Column(name = "id")
18     @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "basket_id_seq")
19     @SequenceGenerator(name = "basket_id_seq", sequenceName = "basket_id_seq", allocationSize = 1)
20     private Integer id;
21     @Column(name = "name_product") no usages
22     String name_product;
23     @Column(name = "price") no usages
24     int price;
25     @Column(name = "swap") no usages
26     int swap;
27 }
```

Рисунок 8 – Листинг кода модели Basket

После создания моделей нужно были реализованы репозитории с одним из способов реализации паттерна DAO[7], а именно через JpaRepository[8], который предоставляет базовые методы по работе с БД. После создания данных интерфейсов (Рисунок 9), этот функционал используется в методах класса сервиса, чтобы выполнять обработку полученных данных и преобразовывать их, например, считать общую стоимость заказа в корзине.


```

1 package com.example.restore.Repositories;
2
3 import com.example.restore.Model.Product;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Modifying;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.data.repository.query.Param;
8 import org.springframework.stereotype.Repository;
9
10 @Repository 2 usages
11 public interface Product_Repository extends JpaRepository<Product, Integer>
12 {
13     @Query("select Bev.name_product from Product Bev where Bev.id = :id") 1 usage
14     String findProductById(@Param("id") int id);
15     @Query("select Bev.price from Product Bev where Bev.id = :id") 1 usage
16     int findPriceById(@Param("id") int id);
17
18
19
20 }

```

Рисунок 9 – Листинг кода для репозитория блюд

Чтобы обеспечить контролер необходимыми данными для вывода, в интерфейсе на Рисунке 9 была использована аннотация Query для получения значений по идентификатору и аннотация Param для передачи параметра id.

2.3 Разработка контролеров и работа с Thymeleaf

Был разработан контролер ProductControler, в котором были использованы такие аннотации как: PostMapping[9] для обработки форм, GetMapping [10] для принятия запросов.

PostMapping отлавливает action запросы [11] от пользователя в форме на клиентской части, после чего выполняет действие по удалению, добавлению, обновлению информации в базе данных. На Рисунке 10 также используется аннотация RequestParam[12] которая позволяет получить значение из текстового поля html в метод в виде параметра.

```

@PostMapping("/add_Product")//Добавление товаров в базу данных Продуктов (Админ панель) по usages
public String addProduct( @RequestParam(name = "name_product_add") String name_product,
                        @RequestParam(name = "price_add") int price,
                        @RequestParam("imageFile") MultipartFile imageFile)
{
    try {
        productService.add(name_product, price, imageFile);
    } catch (IOException e) {
        e.printStackTrace();
        return "error"; // возвращаем страницу ошибки в случае ошибки
    }
    return "redirect:/AdminPage";
}

```

Рисунок 10 – Аннотация PostMapping.

Контролер GetMapping реализует возвращение файла с расширением html на определённый url, также благодаря данной аннотации списки и другие данные добавляются в тег с помощью Thymeleaf[13,14] кода на Рисунке 11, который и выводит данные на клиентскую часть.

```

<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>name_product</th>
      <th>price</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="product : ${products}">
      <td th:text="${product.id}"></td>
      <td th:text="${product.name_product}"></td>
      <td th:text="${product.price}"></td>
    </tr>
  </tbody>
</table>

```

Рисунок 11 – HTML файл с Thymeleaf

Как можем видеть из Рисунка 11, в теге tr используется thymeleaf, который получает из данные из модели [15], в которую было осуществлено добавление из метода с аннотацией GetMapping на Рисунке 12. Также на Рисунках 13, 14 представлены маппинги для других страниц сервиса.

```

@GetMapping("/") //первоначальная страница с добавлением атрибутов для вывода товаров
public String index(Model model)
{
    List<Product> products2 = productService.getAll();
    model.addAttribute( attributeName: "products", products2);
    model.addAttribute( attributeName: "id", attributeValue: 0);
    return "MainPage";
}

```

Рисунок 12 – Аннотация GetMapping главной страницы

```

@GetMapping("/AdminPage") no usages
public String getProducts(Model model) {
    List<Product> products2 = productService.getAll();
    model.addAttribute( attributeName: "products", products2);
    model.addAttribute( attributeName: "id", attributeValue: 0);

    return "AdminPage";
}

```

Рисунок 13 – Аннотация GetMapping страницы админской панели

```

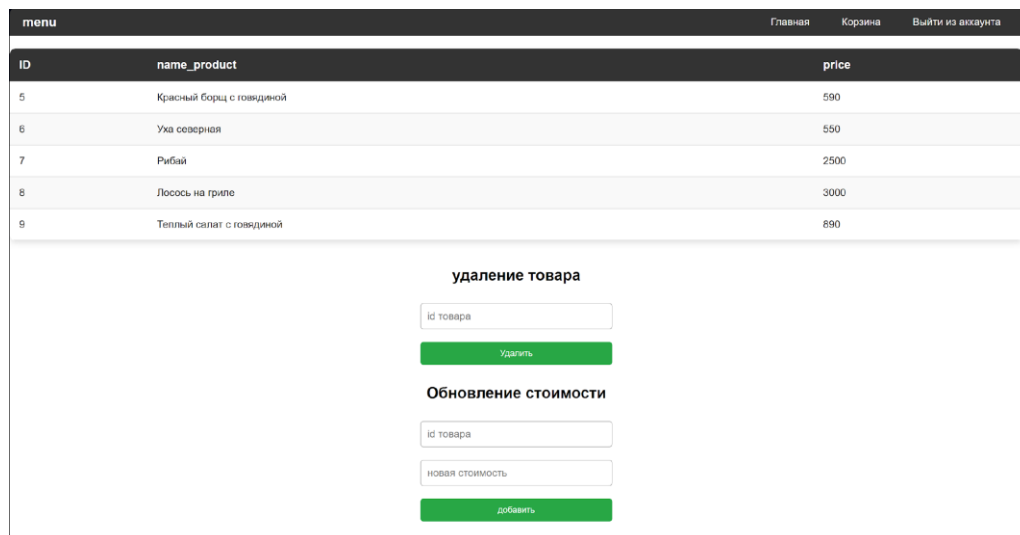
@GetMapping("/basket") // страница корзины с добавлением атрибутов для вывода корзины по
public String getBasket(Model model) {
    List<Basket> baskets = basketService.getAll();
    baskets.sort(Comparator.comparing(Basket::getId)); //сортировка корзины по id товара
    model.addAttribute( attributeName: "baskets", baskets);
    model.addAttribute( attributeName: "id", attributeValue: 0);
    model.addAttribute( attributeName: "full_price", basketService.FullPrice());
    return "BasketPage";
}

```

Рисунок 14 – Аннотация GetMapping страницы корзины

3 ТЕСТИРОВАНИЕ

Было проведено тестирование функций, которое показывает работу с БД (Рисунок 15). Для начала попробуем удалить строку с id 6 и обновить цену для id с номером 7 и получим результат, представленный на Рисунке 16.



The screenshot shows a web application interface for managing a menu. At the top, there is a navigation bar with links: 'menu', 'Главная', 'Корзина', and 'Выйти из аккаунта'. Below the navigation bar is a table with the following data:

ID	name_product	price
5	Красный борщ с говядиной	590
6	Уха северная	550
7	Рибай	2500
8	Лосось на гриле	3000
9	Теплый салат с говядиной	890

Below the table, there are two forms for managing the menu items:

удаление товара

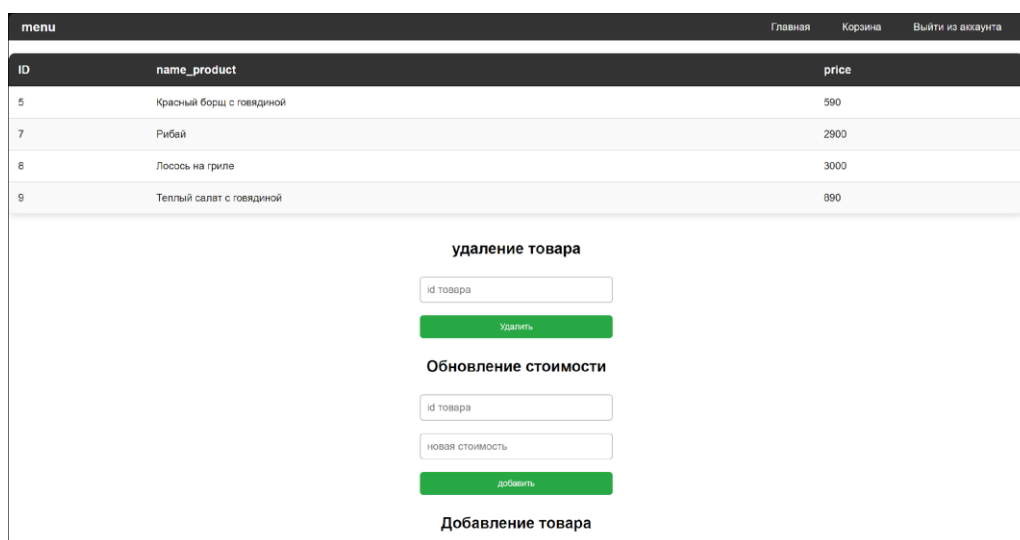
id товара:

Обновление стоимости

id товара:

новая стоимость:

Рисунок 15 – Клиентская часть до взаимодействия



The screenshot shows the same web application interface as Figure 15, but with changes to the menu items. The table now shows:

ID	name_product	price
5	Красный борщ с говядиной	590
7	Рибай	2900
8	Лосось на гриле	3000
9	Теплый салат с говядиной	890

The 'удаление товара' form is still present. The 'Обновление стоимости' form is also present. Below the forms, there is a new form for adding a new item:

Добавление товара

Рисунок 16 – Клиентская часть после взаимодействия

Теперь проведем тестирование корзины БД, для этого добавим несколько блюд в корзину (Рисунок 17).

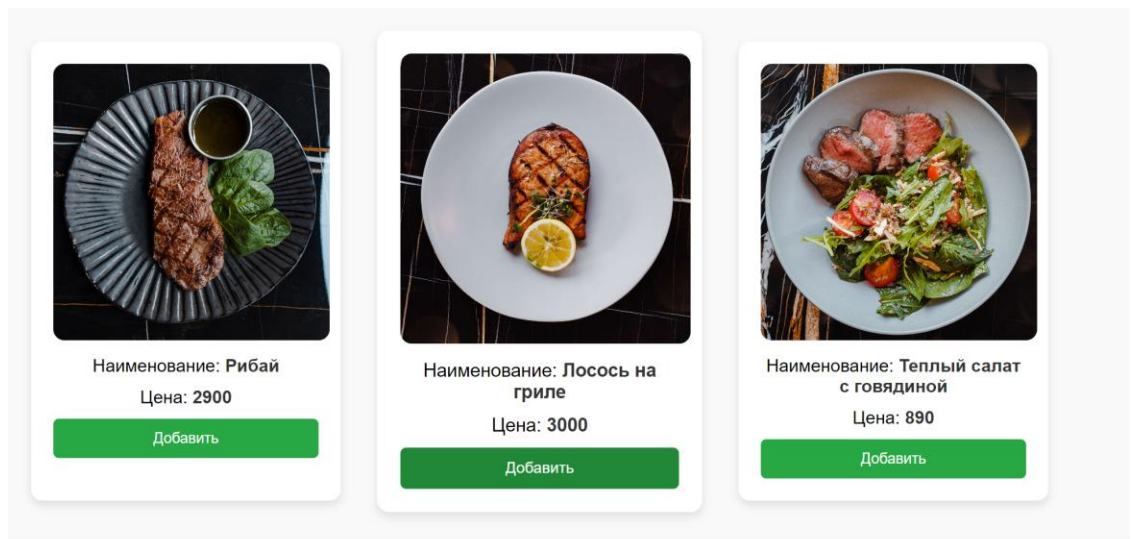


Рисунок 17 – Добавление в корзину

После добавления блюд, переходим на страницу с корзиной, как представлено на Рисунке 18, где данные были добавлены и в случае чего могут быть удалены по нажатию кнопки.

меню					Главная	АдминПанель	Выйти из аккаунта
ID	Товар	Стоимость	Количество	Действия			
2	Лосось на гриле	3000	1	Добавить Удалить			
3	Теплый салат с говядиной	890	1	Добавить Удалить			
Общая стоимость товаров:							
3890							
Заказать товары							

Рисунок 18 – Корзина с заказом

По итогам тестирования весь функционал работает в полной мере.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были проанализированы аналоги веб-приложений, по итогам которых сформирован вывод о нужной функциональности. Во время работы, были усовершенствованы и получены навыки владения современными паттернами и технологиями.

Итогом курсовой работы является веб-приложение «Ресторан», страницы которого реализованы с помощью HTML и CSS, а серверная часть с помощью Spring. В работе с БД были использованы DAO и ORM и настроено взаимодействие с шаблоном Thymeleaf.

Были выполнены все поставленные задачи курсовой работы и разработано приложение.

Исходный код приложения доступен по ссылке - <https://github.com/bevalerry/restore>

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы CSS [Электронный ресурс]. – URL: https://developer.mozilla.org/ru/docs/Learn/Getting_started_with_the_web/CSS_basics (дата обращения 04.04.2023)
2. Основы HTML [Электронный ресурс]. – URL: https://developer.mozilla.org/ru/docs/Learn/Getting_started_with_the_web/HTML_basics (дата обращения 04.04.2023)
3. Spring MVC [Электронный ресурс]. – URL: <https://docs.spring.io/spring-framework/docs/3.2.x/spring-framework-reference/html/mvc.html> (дата обращения 06.04.2023)
4. Lombok [Электронный ресурс]. – URL: <https://javarush.com/groups/posts/3124-spring--ehto-ne-strashno-koroche-lombok> (дата обращения 04.04.2023)
5. PostgreSQL [Электронный ресурс]. – URL: <https://www.postgresql.org/about/> (дата обращения 02.04.2023)
6. Работа с БД [Электронный ресурс]. – URL: <https://javastudy.ru/spring-data-jpa/annotation-persistence/> (дата обращения 04.04.2023)
7. Spring JPA Query [Электронный ресурс]. – URL: <https://habr.com/ru/articles/262243/> (дата обращения 04.04.2023)
8. JPA Repository [Электронный ресурс]. – URL: <https://docs.spring.io/spring-data/jpa/docs/current/api/org/springframework/data/jpa/repository/JpaRepository.html> (дата обращения 07.03.2023)

9. PostMapping [Электронный ресурс]. – URL: <https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/bind/annotation/PostMapping.html> (дата обращения 04.04.2023)

10. GetMapping [Электронный ресурс]. – URL: <https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/bind/annotation/GetMapping.html> (дата обращения 04.04.2023)

11. Обработка action запросов [Электронный ресурс]. – URL: <http://spring-projects.ru/guides/handling-form-submission/> (дата обращения 04.04.2023)

12. RequestParam [Электронный ресурс]. – URL: <https://www.baeldung.com/spring-request-param> (дата обращения 04.04.2023)

13. Thymeleaf часть 1 [Электронный ресурс]. – URL: <https://habr.com/ru/articles/435062/> (дата обращения 04.04.2023)

14. Thymeleaf часть 2 [Электронный ресурс]. – URL: <https://habr.com/ru/articles/435080/> (дата обращения 04.04.2023)

15. Интерфейс Model [Электронный ресурс]. – URL: <https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/ui/Model.html> (дата обращения 04.04.2023)